

TaxaLikely

Most taxonomic assignments rely on match scores (often percent similarity). This makes it easy to confuse a similarity score with the probability that a match is correct. But a 99% similarity between an observation and a reference is not the same as a 99% chance that the observation is that species, because several species may be 99% similar. To link scores to probabilities, matches must first be converted to likelihoods. TaxaLikely does this by fitting statistical models that compare within-species scores to between-species scores in a reference library. In addition to converting scores to likelihoods, this step clarifies how well species can be distinguished using markers. TaxaLikely also models the expected likelihoods of species that are plausible at the site, but absent from the reference library, helping to reduce false negatives. By replacing arbitrary score thresholds with continuous likelihoods, TaxaLikely avoids both overconfident assignments and unnecessary loss of taxonomic resolution. Calibrated likelihoods can then be multiplied by priors (see TaxaExpect) to generate Bayesian posterior probabilities for each hypothesized assignment. Part of the TaxaID ecosystem.

Overview

TaxaLikely trains a hierarchical model on reference-vs-reference match scores (DNA percent identity, image similarity, acoustic scores) and applies it to convert per-observation scores into likelihoods. Supports three hypothesis types:

- **H1 (Known species)** – target taxon is in the reference database
- **H2 (Unreferenced species)** – a congener absent from the reference
- **H3 (Unreferenced genus)** – a taxon from a different genus entirely

Installation

```
# Requires TaxaTools (foundation package)
devtools::install("path/to/TaxaTools")
devtools::install("path/to/TaxaLikely")

# Bioconductor (Gentleman et al. 2004) dependency for reference matrix
# building -- DECIPHER (Wright 2016)
BiocManager::install("DECIPHER")
```

Quick Start

```
library(TaxaLikely)

# 1. Fetch reference sequences from NCBI (National Center for
# Biotechnology Information, U.S. National Library of Medicine,
# National Institutes of Health, Bethesda, Maryland)
reference_df <- fetch_ncbi_reference_sequences(
  taxa = c("Fundulidae", "Gobiidae"),
  barcode_term = "12S",
  rank = "family"
)

# 2. Build pairwise distance matrix
```

```

ref_matrix <- build_sequence_matrix(reference_df)

# 3. Screen for mislabeled references (TaxaMatch) -- see "Detecting Mislabeled
#       References" below for the full pattern
# 4. Train the likelihood model
model <- train_likelihood_model(ref_matrix)

# 5. Apply model to match data (from TaxaMatch)
result <- evaluate_likelihoods(match_df, model)
likelihoods <- result$likelihoods
# Columns: observation_id, taxon_name, hypothesis_type,
#           score_likelihood, score_likelihood_mean, score_likelihood_sd,
#           score_likelihood_cov, score_likelihood_evidence, h2_delta_source

```

Loading Pre-built Reference Databases

TaxaLikely supports several common reference database formats directly.

CRABS (recommended for eDNA barcode databases)

CRABS (Creating Reference databases for Amplicon-Based Sequencing) is a widely-used eDNA reference database builder. Its internal format is a headerless tab-delimited file with 11 fixed columns (accession through sequence). Load it with:

```

ref <- read_crabs_output(
  crabs_file = "mifish_12S_crabs.tsv",
  rank_system = c("family", "genus", "species"), # or NULL to auto-detect
  max_n_bases = 250, # optional: drop unusually long sequences
  dereplicate = TRUE # optional: collapse identical seqs within species
)

```

CRABS + TaxaLikely are complementary. CRABS excels at bulk retrieval, length filtering, primer trimming, and exact-sequence dereplication at database-build time. Neither CRABS nor TaxaLikely catches mislabeled sequences where the species annotation is wrong but the sequence itself is valid – these produce within-species distances that look like between-species distances and inflate false-positive rates. Screening for this lives in TaxaMatch, not TaxaLikely:

```

ref_matrix <- build_sequence_matrix(ref)
model <- train_likelihood_model(ref_matrix)

```

See Detecting Mislabeled References below for the recommended screening step – run it on `ref` before `build_sequence_matrix()`, not after.

FASTA + separate taxonomy table

`read_reference_fasta()` accepts a FASTA file and a separate taxonomy source. Two input formats are supported:

Data frame (custom databases, CRUX, GenBank dumps):

```
tax <- data.frame(
  composite_id = c("ACC001", "ACC002"),
  family = c("Fundulidae", "Atherinopsidae"),
  genus = c("Fundulus", "Atherinops"),
  species = c("Fundulus parvipinnis", "Atherinops affinis")
)
ref <- read_reference_fasta("sequences.fasta", taxonomy = tax,
  rank_system = c("family", "genus", "species"))
```

Taxonomy TSV file (QIIME2 / RESCRIPT / SILVA / MIDORI2): supply `taxonomy_file` instead of `taxonomy`. Both prefix-style (`k__Kingdom;p__Phylum;...`) and positional (`Kingdom;Phylum;...`) formats are auto-detected. Header rows are skipped automatically.

```
# Works with QIIME2 taxonomy artifacts, RESCRIPT output, or MIDORI2 files
ref <- read_reference_fasta(
  "sequences.fasta",
  rank_system = c("family", "genus", "species"),
  taxonomy_file = "taxonomy.tsv"
)
```

Reference Databases at a Glance

The table below summarises the databases commonly used with TaxaLikely, the data types they cover, and the recommended loading path.

Database	Focus	Typical size	Loading path	Notes
NCBI GenBank	Universal	API (no local file)	<code>fetch_ncbi_reference_sequences()</code>	Per-sequence API query; best for targeted eDNA marker retrieval
CRABS output	eDNA amplicons	Varies	<code>read_crabs_output()</code>	CRABS handles bulk QC; TaxaLikely adds mislabel detection
SILVA SSU	16S / 18S / 23S rRNA	~1.3 GB	<code>subset_local_database()</code>	Primary database for microbial amplicon eDNA; ~510 k sequences
MIDORI2	COI + nuclear markers	~4.4 GB (COI)	<code>subset_local_database()</code>	Best for metazoan COI and nuclear eDNA markers
GTDB	Bacterial / archaeal 16S	~500 MB (16S subset)	<code>subset_local_database()</code>	Prokaryotic genomic taxonomy; differs from NCBI; export via QIIME 2
Greengenes2	16S rRNA	~1.2 GB	<code>subset_local_database()</code>	2012 release; GTDB-derived taxonomy; export from QIIME 2

Database	Focus	Typical size	Loading path	Notes
RDP	16S / 28S rRNA	~200 MB	<code>read_reference_fasta()</code>	Smaller than SILVA; reformat lineage file to 2-col TSV first

SILVA, MIDORI2, GTDB, and Greengenes2 are distributed as bulk downloads (multi-gigabyte FASTA + taxonomy files). `subset_local_database()` streams these files rather than loading them into memory, so filtering to the genera or families relevant to your site is fast regardless of total database size. CRABS can also download from SILVA and BOLD directly and produce its own internal format, which `read_crabs_output()` handles. For NCBI, use `fetch_ncbi_reference_sequences()` to fetch only the taxa you need.

Subsetting a Large Local Database

`subset_local_database()` filters any large FASTA + taxonomy file to a user-supplied taxon list. The taxonomy file is parsed first to identify matching sequence IDs; the FASTA is then streamed, so peak memory scales with the number of matching sequences rather than the total database size.

```
library(TaxaLikely)

# --- SILVA SSU example: filter to two fish families -----
ref <- subset_local_database(
  fasta_path = "SILVA_138.1_SSURef_NR99.fasta.gz",
  taxa       = c("Fundulidae", "Gobiidae"),
  rank       = "family",
  rank_system = c("family", "genus", "species"),
  taxonomy_file = "silva_taxonomy.tsv"
)

# --- MIDORI2 COI example: filter to a genus; discard sequences > 700 bp -----
ref <- subset_local_database(
  fasta_path = "MIDORI2_UNIQ_NUC_GB260_COI_QIIME.fasta",
  taxa       = "Thunnus",
  rank       = "genus",
  rank_system = c("family", "genus", "species"),
  taxonomy_file = "MIDORI2_UNIQ_NUC_GB260_COI_QIIME_taxon.tsv",
  max_n_bases = 700L,
  require_species = TRUE
)

# Both return a reference_df - continue with the standard workflow (screen
# ref for mislabeled accessions via TaxaMatch first -- see "Detecting
# Mislabeled References" below):
ref_matrix <- build_sequence_matrix(ref)
model      <- train_likelihood_model(ref_matrix)
```

`subset_local_database()` accepts plain-text and .gz-compressed FASTA files. Taxonomy strings in SILVA, MIDORI2, GTDB, and Greengenes2 prefix (d__, p__, ...) or positional (no-prefix semicolon) formats are both auto-detected. For GTDB or Greengenes2 sources, export the FASTA + taxonomy TSV from QIIME 2 with `qiime tools export` before calling the function.

Building a Site-Specific Reference Library

Build a curated local reference matched to your site's expected taxa by chaining three functions directly: fetch sequences from NCBI, audit which described species have no barcode sequences at all (the unreferenced gap list), then export a FASTA + taxonomy TSV ready for BLAST or QIIME 2.

```
library(TaxaLikely)

# Genera expected at your site (from TaxaExpect or user-supplied)
site_genera <- c("Fundulus", "Gambusia", "Lepomis", "Micropterus")

reference_df <- fetch_ncbi_reference_sequences(
  taxa      = site_genera,
  barcode_term = "MiFishU",
  max_date   = "2024/12/31"          # reproducible: fix GenBank state
)

coverage <- audit_barcode_coverage(reference_df, barcode_term = "MiFishU")
coverage$unreferenced # species with NO barcode in NCBI -- cannot be detected
coverage$census       # per-genus completeness summary

# Pass gap list to the LLM-shortcut pipeline for ghost hypothesis handling
llm_posteriors <- TaxaAssign::assign_taxa_llm(
  match_df      = match_data,
  unreferenced_taxa = coverage$unreferenced
)

write_reference_fasta(
  reference_df,
  file      = "site_reference/reference.fasta",
  taxonomy_file = "site_reference/reference_taxonomy.tsv" # QIIME2-compatible; reload with read_referen
)

# Train model on the curated reference
model <- train_likelihood_model(
  build_sequence_matrix(reference_df)
)
```

Key Functions

Reference acquisition and export (DNA): - `subset_local_database()` – stream a large local FASTA + taxonomy file (SILVA, MIDORI2, GTDB, Greengenes2) and extract sequences for a user-supplied taxon list; memory scales with matches, not database size - `write_reference_fasta()` – export any `reference_df` to FASTA + optional taxonomy TSV (round-trippable with `read_reference_fasta()`) - `read_crabs_output()` – load a CRABS internal-format database (taxonomy embedded; no separate file needed) - `fetch_ncbi_reference_sequences()` – download from NCBI by taxon + barcode marker - `read_reference_fasta()` – load local FASTA + data-frame taxonomy (or `taxonomy_file` TSV for QIIME2/RESRIPT/MIDORI2) - `trim_to_amplicon()` – in-silico PCR: extract just the amplicon region from an over-length sequence (e.g. a full mitogenome swept up by an NCBI fetch) via primer matching, instead of discarding it outright; run between `fetch_ncbi_reference_sequences()`/ `read_reference_fasta()` and `build_sequence_matrix()`. Primer pairs come from `TaxaTools::barcode_primer_defaults` – see `?barcode_primer_defaults` (in `TaxaTools`) for the full list of verified primer sets and their known

limitations (e.g. `coi-folmer`’s documented vertebrate mismatches, `coi-leray`’s reduced discriminatory power relative to full-length COI) before trusting a result on real data.

Model training (DNA): - `build_sequence_matrix()` – pairwise distance matrix via DECIPHER; required for `train_likelihood_model()` (screen for mislabeled references via TaxaMatch first – see “Detecting Mislabeled References” below) - `train_likelihood_model()` – fit hierarchical Bayesian model (pair-coverage floor and empirical Bayes shrinkage on by default; see “Reference Coverage Quality Filtering” and “Statistical Design” below)

Inference: - `calibrate_query_noise()` – correct H1’s mean for query-vs-reference technical noise invisible to reference-vs-reference training pairs; optional `evidence_col` also establishes a baseline for `evaluate_likelihoods(evidence_col=)`’s gated sigma rescale - `evaluate_likelihoods()` – convert match scores to likelihoods using a trained model - `filter_top_hypotheses()` – keep finest-rank candidates per query - `unreferenced_candidates()` – expand a consensus assignment with H2/H3 placeholder rows (no model required; used in no-score and acoustic/image pathways) - `assign_scores()` – set `score_likelihood` values: uniform (`"none"`), ratio-normalised (`"probability"`), softmax (`"similarity_softmax"`), or prepare for the bivariate-normal model (`"similarity"`) – for the DNA scored pathway, real callers train a model (`train_likelihood_model()`) and call `evaluate_likelihoods()` directly on the match object rather than continuing from `assign_scores()`’s `"similarity"` output - `expand_unreferenced_hypotheses()` – models likelihoods for named unreferenced species (borrowed from the generic H2/H3 values) and expands them so they can join TaxaExpect priors directly; requires a TaxaExpect- derived unreferenced-species list, so it runs after both TaxaLikely and TaxaExpect and before `TaxaAssign::compute_posterior()` (moved from TaxaAssign in September 2026)

Reference QC: - `audit_barcode_coverage()` – find unreferenced species (no barcode sequence; eDNA/DNA only) - `suggest_unreferenced_species()` – fast, LLM-first alternative to `audit_barcode_coverage()` (also supports acoustic/image via `data_type`); feeds `TaxaAssign::assign_taxa_llm(unreferenced_taxa = ...)`, the LLM-shortcut pathway – moved here from TaxaAssign in September 2026 - `audit_acoustic_coverage()` – find plausible species absent from classifier’s known list (acoustic/image) - `audit_reference_coverage()` – taxonomic completeness check - `apply_coverage_constraints()` – suppress H2 for fully-sampled genera. Mislabel screening and match-object cleaning for flagged accessions live in TaxaMatch, not here – see “Detecting Mislabeled References” below.

Diagnostics and reporting: - `interpret_model()` – summarize trained model parameters - `report_likelihood()` – generate report section for `assemble_report()`

Detecting Mislabeled References

TaxaLikely has no built-in reference-quality screening. `train_likelihood_model()` trains on whatever `raw_df/ref_matrix` it’s given. To avoid entering mislabeled references into the training set, screen it first, upstream, via TaxaMatch’s **Reference Accession Quality** section.

Data types other than DNA sequences

Reference source	Screen for mislabeling?	Notes
NCBI nucleotide (via <code>fetch_ncbi_reference_sequences()</code>)	Yes, using the pattern above	NCBI has well-known curation issues: automated submissions, misidentified vouchers, contamination.
Curated libraries (CRUX, custom expert-built FASTA)	Optional	Lower mislabeling rate than NCBI. If your library has a quality column, use that filter instead.

Reference source	Screen for mislabeling?	Notes
Xeno-canto * bird sounds (acoustic)	No	Xeno-canto is expert-curated; species identity mislabeling is rare. The dominant noise source is recording conditions (distance, background), not wrong species. Filter on the recording's own quality grade (A-E) instead – the acoustic <code>seq_matrix</code> 's <code>coverage</code> column encodes quality grade categorically; a dedicated threshold-calibration helper for this existed here and was archived after a real A/B test found the accuracy gain came with a real coverage cost – apply a threshold directly (e.g. <code>seq_matrix[seq_matrix\$coverage >= threshold,]</code>) if you need one.
Camera trap images (Animl/SpeciesNet)	Not applicable	This screen is DNA-specific (BLAST + sequence alignment). Camera trap ground-truth labeling has different error modes (occlusion, blur, multiple animals, handler setup) needing a different mechanism, not built.

* Xeno-canto (Xeno-canto Foundation, Netherlands, with support from Naturalis Biodiversity Center, Leiden; <https://xeno-canto.org/>).

Reference Coverage Quality Filtering

Match scores (`p_match`) measure how similar two sequences or audio clips are, but they do not capture *how much* of each observation contributed to that score. A 99% DNA identity computed from a 50 bp fragment of a 600 bp barcode is far less reliable than the same identity computed from a 580 bp overlap — yet both produce the same score.

`build_sequence_matrix()` attaches a `coverage` column to its output: *alignment coverage* is the number of positions where both sequences contribute a non-gap character, divided by the shorter unaligned sequence length. Values near 1.0 indicate nearly complete overlap; values near 0.0 indicate highly gappy or partial alignments.

`train_likelihood_model()` uses that column itself (2026-09-10): its `min_pair_coverage` floor (default 0.8) decides which pairs may *define* a reference's best foreign, best congener and best conspecific match. It removes no pair from the data and drops no species – a reference with no conspecific pair above the floor falls back to its best one. The floor exists because a pairwise alignment between a short deposit and an unrelated sequence can be 100% identical over a few percent of the sequence; on real 12S data that was the “best foreign match” for 86% of references, so the trained gap was negative for 89% of them while every real BLAST candidate at inference had coverage of at least 80. The floor must equal the coverage floor the match object was built under (`TaxaMatch::blast_sequences(min_query_coverage = 80)` is 0.8); `blast_sequences()` records that value in the match object and `evaluate_likelihoods()` warns if the two disagree.

```
ref_matrix <- build_sequence_matrix(reference_df)
model <- train_likelihood_model(ref_matrix, min_pair_coverage = 0.8)
```

Do not confuse this with excluding low-coverage pairs from training altogether. Be cautious about that, even manually: a real test on full-scale production data found that a coverage floor produces a genuine likelihood-quality improvement on the pairs it keeps, but also a real, quantified cost — roughly 19% of species can lose every training pair, and roughly 25% of real evaluation queries end up unresolved. Hard exclusion on an imperfect quality proxy carries this risk generally, not just for coverage (see `apply_coverage_constraints()` and `TaxaFetch::filter_gbif_quality()`, both of which flag rather than exclude for the same reason). If you filter by coverage, check whether the excluded pairs are a random cross-section or systematically concentrated in particular species before trusting the result.

Statistical Design

TaxaLikely models the joint distribution of two features: the transformed match score (absolute fit) and the gap to the best alternative (relative uniqueness), as a bivariate normal for each hypothesis type:

- **Score + gap features:** Raw scores are transformed to an unbounded (or near-unbounded) domain; the gap is computed on the same scale so that differences near 100% are amplified appropriately
- **Transform choice (`score_transform`):** "logit" (default, $\ln(p/(1-p))$) or "sqrt_mismatch" ($-\sqrt{1-p}$, Anscombe's (1948) classical rare-event-count variance stabilizer, applied to the match *mismatch*). Logit's derivative diverges fastest exactly where real barcode matches concentrate (near 100% identity), which was found to reverse the sign of genus-tightness comparisons for the H2/H3 unreferenced-relative hypotheses below. This was confirmed on real 12S congener data (Pearson $r = +0.465$ under logit vs. the correct $r = -0.55$ on the raw proportion scale). `sqrt_mismatch` recovers the correct direction ($r = -0.235$) and is opt-in; H1/H2/H3 always share one transform (mixing them would require an explicit change-of-variables correction that is not implemented). See `inst/TaxaLikely_supplemental_methods.md` Section 3A-i for the full derivation and validation.
- **Bivariate normal likelihood:** The joint (score, gap) density captures interactions; small gap is more tolerable when the score is very high
- **Empirical Bayes shrinkage:** Per-species parameters are shrunk toward the global mean (Efron and Morris 1973), with weight inversely proportional to sample size, preventing poorly sampled species from having unreliable estimates
- **Per-species sigma floor:** At inference, the per-species `sigma_score` from `H1_Lookup` is floored at the global `H1_Sigma[1,1]`. Reference-vs-reference training pairs for well-sampled species can be artificially tight (many near-identical NCBI accessions from the same voucher), producing a sigma smaller than the global estimate. Real query-vs-reference scores span a wider range due to intraspecific variation and sequencing error. The floor ensures those species still receive non-zero H1 likelihoods at realistic eDNA query scores
- **Score-only outlier filter (`alpha = 0.001`):** Before computing H1 density, `.evaluate_one_query()` tests whether the query score is consistent with the H1 species distribution using a univariate chi-squared test ($df = 1$, score only). If the p-value is below `alpha`, the candidate is treated as a score outlier and receives likelihood 0. The gap feature is deliberately excluded from this test: a small gap (a closely-scoring congener is present) is correctly handled by the bivariate density, which returns a lower H1 value in that case. Including the gap in the outlier test would incorrectly reject legitimate H1 candidates in species-rich families where confusable congeners exist. The default `alpha = 0.001` (~3.3 sigma, 1-in-1000) drops genuinely inconsistent cross-family BLAST artefacts (e.g. freshwater Cyprinidae at 91-93% identity in a marine sample, >4 sigma from H1 mean) while retaining legitimate borderline H1s (e.g. a coastal species at 99% identity, ~2.7 sigma from H1 mean, $p = 0.006$). This test is **H1-intrinsic** – no comparison to H2/H3 densities

- **H2/H3 offset distributions:** Unreferenced species and genus hypotheses use the H1 distribution shifted left by learned delta offsets, estimated from cross-species match scores in training data. The mean anchors on the specific best-matching referenced species' own resolved mean (not just the population-wide average), and where a genus has a real congener pair, `H2_Lookup` supplies a genus-specific delta and variance shrunk toward the pooled default, rather than every genus sharing one pooled offset and width regardless of how tightly or loosely its species cluster. `evaluate_likelihoods()` marks each H2/H3 row with `h2_delta_source` ("genus_specific" or "global_fallback") so a row using the cruder pooled approximation can be identified.
- **Perfect-match anchoring:** Synthetic 100% match pseudo-data prevent the “perfection penalty” where the Gaussian density peaks below 100%
- **Query-vs-reference noise calibration (`calibrate_query_noise()`):** `train_likelihood_model()` estimates H1 entirely from reference-vs-reference pairs (two clean, curated database accessions compared to each other), which carries none of the technical noise (PCR/sequencing/degradation/ ASV-inference) a real query picks up – so a genuinely correct match routinely scores below the trained H1 mean and loses to the unreferenced hypotheses. `calibrate_query_noise()` estimates one marker-wide additive offset from observations whose species can be identified with high confidence from independent occurrence-prior data (non-circular, since it never touches the match scores or likelihood model being calibrated) and shifts `H1_Global_Mu/H1_Lookup$mu_score` uniformly; H2/H3 move automatically since their means are defined relative to H1's. This offset does **not** transfer between markers or datasets and must be re-estimated for each. An optional `evidence_col` (e.g. real per-observation DNA read depth) additionally establishes a baseline that `evaluate_likelihoods(evidence_col=)` uses to selectively widen H1 sigma for lower-evidence observations, via a closed-form crossover gate (only applied when doing so is provably non-decreasing for that candidate's density) rather than an unconditional rescale.
- **Monte Carlo uncertainty:** Each candidate's *trained mean* (not the query's own fixed, already-known observed score) is perturbed across simulations by its shrinkage-consistent estimation uncertainty ($\text{Var}(\mu) \approx w^2 * \sigma^2 / N$, using the same Empirical Bayes weight `w` as the point estimate), and the query's real observed point is re-evaluated against each draw. This yields `score_likelihood_mean` and `score_likelihood_sd`, measuring how confidently the candidate's own parameters are known – H2/H3, which borrow a shifted mean rather than observing their own species directly, correctly come out wider than a well-referenced H1 candidate.
- **Pair-coverage floor (`min_pair_coverage`, default 0.8):** only a reference pair at or above the match object's own coverage floor may define a reference's best foreign/congener/conspecific match when the gap feature is trained. Train/inference distribution matching for one feature, not a data filter – no pair removed, no species dropped. Validated on the real GreatLakes workflow code path against the independent Lamar species list: species co-detections 593 to 798, precision 0.805 to 0.818, 29 to 41 of 61 species, none lost.
- **Shrinkage weights (`shrinkage`, default "empirical_bayes"):** each species' H1 mean score and mean gap are shrunk toward the global mean with weight $\tau^2 / (\tau^2 + \sigma^2/N)$, where τ^2 (the real between-species variance of the means) is estimated by method of moments, per dimension. No signal beyond noise collapses every species to the global mean; real signal lets well-referenced species keep more of their own. On real 12S data the score means carry no such signal (weight 0 for every species) while the gap means do (weights 0.34-0.72). "fixed" restores the original $N / (N + \text{prior_weight})$ weight exactly. Variance shrinkage always uses the fixed weight.
- **Alignment coverage filter (optional):** A `min_coverage` threshold can be passed to `evaluate_likelihoods()` to drop low-coverage candidates before scoring. Acoustic and image data can supply a categorical quality column (e.g. Xeno-canto grade) encoded as an integer for the same pre-filter. See “Reference Coverage Quality Filtering” above for the archival history of the dedicated threshold-calibration helpers this section used to point at.

- **Coverage-adjusted likelihood (`score_likelihood_cov`):** When the match object contains a coverage column, `evaluate_likelihoods()` produces a second point-estimate column `score_likelihood_cov` alongside the standard `score_likelihood`. For each H1 candidate the model sigma is widened by $1 / \sqrt{\text{coverage}}$ before the likelihood is evaluated — grounded in binomial sampling theory: the standard error of a proportion estimated from N aligned positions scales as $1 / \sqrt{N}$, and $N_{\text{aligned}} = \text{coverage} \times N_{\text{total}}$. H2 and H3 sigmas are global fixed parameters and are not inflated.

Widening sigma changes the **point estimate** of the likelihood (not its SD), because it changes the density value returned by the bivariate normal at the observed score. The direction of the change depends on where the observed score falls relative to the H1 mean:

- **Score near the H1 mean (good match):** the density at the peak decreases as the distribution flattens $\rightarrow \text{score_likelihood_cov} < \text{score_likelihood}$. This is the primary intended effect: a 98% match at 60% coverage is penalised relative to the same score at full coverage, reducing overconfidence in partial alignments.
- **Score far below the H1 mean (poor cross-species match):** the distribution flattens into a wider tail, increasing the density at that low score $\rightarrow \text{score_likelihood_cov} > \text{score_likelihood}$. This is a secondary effect: the model is saying “with low coverage I cannot rule out H1 as strongly.” These candidates are already ranked low and the effect rarely changes downstream assignments.

The crossover is at exactly ± 1 sigma from the mean — a useful check is whether the best H1 candidate is a good match (expect negative delta) or a poor match (expect positive delta).

Why this is not a model parameter: Reference-vs-reference training pairs (from `build_sequence_matrix()`) are nearly always full-length alignments — same-species sequences share the same amplicon and align completely. There is therefore no within-H1 coverage variation in the training data from which to estimate a coverage-score relationship. The $1 / \sqrt{\text{coverage}}$ inflation is applied as a principled prior at inference time rather than a fitted parameter.

How to use it: Pass `score_likelihood_cov` to `TaxaAssign::compute_posterior()` instead of `score_likelihood` to apply the adjustment. Compare the two columns to find queries where coverage meaningfully shifts the likelihood ratios:

```
likelihoods |>
  dplyr::filter(
    hypothesis_type == "specific_candidate",
    abs(score_likelihood_cov - score_likelihood) > 0.05
  ) |>
  dplyr::arrange(dplyr::desc(abs(score_likelihood_cov - score_likelihood)))
```

When coverage is absent from the match object or equals 1 for all candidates, `score_likelihood_cov` is identical to `score_likelihood`.

For a detailed treatment of the statistical framework, feature engineering, hypothesis definitions, parameter estimation, and reference quality control, see `inst/TaxaLikely_supplemental_methods.md`.

Acoustic and Image Workflows

For acoustic (BirdNET) and image (SpeciesNet, Animl, iNaturalist CV) data, TaxaLikely works as a **post-classifier** layer. Users bring classifier output files that already contain candidates and confidence scores — TaxaLikely converts those scores to likelihoods and expands the candidate set using TaxaExpect priors.

There are two entry points depending on how many candidates the classifier provides per observation:

Multiple candidates per observation (e.g., SpeciesNet top-k)

When the classifier returns ranked candidate lists (top-k species with scores), the match data maps directly to the scored pathway. Use `evaluate_likelihoods()` exactly as in the DNA workflow:

```
library(TaxaLikely)
library(TaxaMatch)

# Read SpeciesNet / Animl top-k output → canonical match object
# (scientific names required; use TaxaTools::common_to_scientific()
# if your classifier outputs common names)
match_df <- read_animl_output("animl_results/", min_confidence = 0.3) |>
  subset(!species %in% c("empty", "human", "vehicle"))

# Check coverage: which plausible species are absent from classifier's
# known species list?
census <- audit_acoustic_coverage(
  plausible_species = c("Ursus americanus", "Puma concolor", "Cervus canadensis"),
  reference_species = speciesnet_species_list
)
census$unreferenced # species the classifier has never seen

# Evaluate likelihoods (requires a trained model or use no-score pathway)
result <- evaluate_likelihoods(match_df, image_model)
likelihoods <- result$likelihoods
```

Single best candidate with a score (e.g., BirdNET top-1)

`assign_scores()` anchors H2/H3 likelihoods at the **median H1 likelihood across all candidates for that observation**. With only one H1 row per observation (top-1 output), that median is always 1.0, so the confidence score has no effect and all three rows receive `score_likelihood = 1.0`.

Recommendation: keep BirdNET's full ranked output (multiple candidates per segment) and use `score_type = "probability"`. BirdNET's default output already includes ranked species with confidence scores:

```
library(TaxaLikely)

# BirdNET full output - multiple candidates per clip, each with confidence
# (use TaxaTools::common_to_scientific() first if output has common names)
birdnet_df <- data.frame(
  observation_id = c("clip_001", "clip_001", "clip_001"),
  taxon_name     = c("Melospiza melodia", "Passerella iliaca", "Junco hyemalis"),
  taxon_name_rank = c("species", "species", "species"),
  family         = c("Passerellidae", "Passerellidae", "Passerellidae"),
  genus          = c("Melospiza", "Passerella", "Junco"),
  species        = c("Melospiza melodia", "Passerella iliaca", "Junco hyemalis"),
  score_original = c(0.87, 0.09, 0.04) # BirdNET confidence scores
)

# Step 1: add H2/H3 placeholder rows
hyp_df <- unreferenced_candidates(birdnet_df,
  rank_system = c("family", "genus", "species"))
```

```
# Step 2: ratio-normalise; top candidate = 1.0, lower ones proportionally less
likelihoods <- assign_scores(hyp_df, score_type = "probability")

# Feed directly to TaxaAssign:
posteriors <- TaxaAssign::compute_posterior(likelihoods, priors_df = my_priors)
```

If you only have top-1 BirdNET output and cannot recover the full ranked list, use `score_type = "none"` — posteriors will be proportional to TaxaExpect priors alone (the confidence score is ignored).

Common names in classifier output

If your classifier outputs common names rather than scientific names, convert them first with `TaxaTools::common_to_scientific()`:

```
library(TaxaTools)

sci <- common_to_scientific(
  common_names = c("White-tailed Deer", "Raccoon", "Wild Turkey"),
  taxon_group   = "mammals and birds",
  location      = "Eastern USA"
)
sci[, c("common_name", "scientific_name_verified", "verified")]
```

No-Score Pathway

When match scores are unavailable, morphology-based identifications, expert IDs, upranked consensus outputs from a previous run, or any source that yields a taxon name but no similarity score, `unreferenced_candidates() + assign_scores(score_type = "none")` builds a degenerate likelihood object that bypasses `TaxaMatch` and `evaluate_likelihoods()` entirely.

Two placeholder rows are added per observation: - **H2 (unreferenced_species)**, a placeholder for any species in the same genus not in the reference database - **H3 (unreferenced_genus)**, a placeholder for any genus in the same family not in the reference database

All `score_likelihood` values are set to 1.0 (uniform), so posteriors are proportional to TaxaExpect priors. No `priors_df` is needed at this stage. Priors are joined later by `TaxaAssign::join_priors()`.

```
library(TaxaLikely)

# Consensus taxon assignments with no match scores
consensus_df <- data.frame(
  observation_id = c("obs1", "obs2", "obs3"),
  taxon_name     = c("Salmo salar", "Salvelinus", "Cyprinus carpio"),
  taxon_name_rank = c("species", "genus", "species"),
  family         = c("Salmonidae", "Salmonidae", "Cyprinidae"),
  genus          = c("Salmo", "Salvelinus", "Cyprinus"),
  species        = c("Salmo salar", NA_character_, "Cyprinus carpio")
)

# Step 1: add H2/H3 placeholder rows
hyp_df <- unreferenced_candidates(
  match_df = consensus_df,
```

```

rank_system = c("family", "genus", "species")
)

# Step 2: set all score_likelihood = 1.0
likelihoods <- assign_scores(hyp_df, score_type = "none")

# Three rows per observation (H1 + H2 + H3); all score_likelihood = 1.0.
# Feed directly to TaxaAssign; posteriors will be proportional to priors:
posteriors <- TaxaAssign::compute_posterior(likelihoods, priors_df = my_priors)

```

When to use this pathway

- Morphology or expert IDs that have no match scores
- Consensus outputs upranked to genus or family (score discrimination failed; use priors to probe which species is most likely)
- Stability checks: does an unreferenced congener have a higher prior than the consensus species? If so, flag for review.
- Mixed datasets: run `evaluate_likelihoods()` on scored observations and `unreferenced_candidates()` + `assign_scores()` on unscored ones, then `dplyr::bind_rows()` the outputs before calling `compute_posterior()`.

See `inst/workflows/6_no_score_pathway_workflow.R` for a full example.

Cache

`fetch_ncbi_reference_sequences()` and `audit_barcode_coverage()` cache to a persistent, per-user directory (`tools::R_user_dir("TaxaLikely", "cache")`) so re-running the same taxon/marker/params combination skips the NCBI fetch. Neither expires automatically.

Run `taxalikely_clear_cache(dry_run = TRUE)` to see how much space the cache is using before clearing it, or `taxalikely_clear_cache()` to clear it directly.

Vignettes

- Score to Likelihood – full workflow

Part of TaxaID

TaxaLikely receives match data from TaxaMatch and produces calibrated likelihoods for TaxaAssign (posterior computation).

Ecosystem: TaxaMatch -> **TaxaLikely** -> TaxaAssign

See the TaxaID README for ecosystem overview and installation instructions.

Citation

Lafferty, K.D., 2026, TaxaID – A modular R ecosystem for Bayesian taxonomic assignment: U.S. Geological Survey software release, <https://doi.org/10.5066/xxxxxx>.

Software Requirements

- R ($\geq 4.1.0$; R Core Team 2025)
- TaxaTools (foundation package, installed first)
- DECIPHER (Wright 2016) and Biostrings (Bioconductor, Gentleman et al. 2004; required for `build_sequence_matrix()` only)
- rentrez and xml2 (for NCBI reference fetching and coverage auditing)

All dependencies are declared in the DESCRIPTION file and installed automatically.

Developed with Claude Code (Anthropic PBC, San Francisco, California).

References

- Anscombe, F.J. (1948). The transformation of Poisson, binomial and negative-binomial data. *Biometrika*, 35(3/4), 246–254.
- Efron, B. and Morris, C. (1973). Stein’s estimation rule and its competitors – an empirical Bayes approach. *Journal of the American Statistical Association*, 68(341), 117–130.
- Gentleman, R.C., Carey, V.J., Bates, D.M., Bolstad, B., Dettling, M., Dudoit, S., Ellis, B., Gautier, L., Ge, Y., Gentry, J., Hornik, K., Hothorn, T., Huber, W., Iacus, S., Irizarry, R., Leisch, F., Li, C., Maechler, M., Rossini, A.J., Sawitzki, G., Smith, C., Smyth, G., Tierney, L., Yang, J.Y.H. and Zhang, J. (2004). Bioconductor: open software development for computational biology and bioinformatics. *Genome Biology*, 5, R80.
- R Core Team (2025). R: A Language and Environment for Statistical Computing. V.4.5.2. R Foundation for Statistical Computing, Vienna, Austria. <https://www.r-project.org>
- Wright, E.S. (2016). Using DECIPHER v2.0 to analyze big biological sequence data in R. *The R Journal*, 8(1), 352–359.